Gorgo Go for Java Programmers

May 29, 2026

Contents

Chapter 19

Reflection

Reflection lets a program inspect and manipulate its own types and values at runtime, without knowing them at compile time. Go's `reflect` package is how serialization libraries, dependency injection frameworks, and testing tools peer inside your structs — but it comes at a real performance cost, so it is a sharp tool to reach for only when compile-time alternatives fall short.

The Three Laws of Reflection

Rob Pike formalized reflection in Go around three laws that govern how values move between the ordinary type system and the `reflect` world. Understanding them prevents the most common mistakes.

Law 1: Reflection goes from interface value to reflection object. When you call `reflect.TypeOf` or `reflect.ValueOf`, you pass a value; the runtime wraps it in an interface and hands you a `reflect.Type` or `reflect.Value` that describes it.

Law 2: Reflection goes from reflection object back to interface value. A `reflect.Value` can be converted back to an `interface{}` with `Value.Interface()`, from which you can type-assert to the concrete type.

Law 3: To modify a reflection object, the value must be settable. Settability is a stricter condition than addressability. If you want to set a field through reflection, you must start with a pointer to the value — not the value itself.

Tip: Think of the three laws as a round trip: `interface{}` -> `reflect.Value` (law 1) -> `interface{}` (law 2) -> concrete type. Mutation requires a pointer from the start (law 3).

If you have used `java.lang.reflect`, the overall structure will feel familiar: you interrogate a type descriptor, walk its fields or methods, and optionally read or write values through accessor objects. The mechanics differ, but the mental model maps cleanly — Go's `reflect.Type` is roughly `java.lang.Class`, and `reflect.Value` is roughly `java.lang.reflect.Field` combined with the value it holds.

`reflect.TypeOf` and `reflect.ValueOf`

The two entry points to the `reflect` package are `TypeOf` and `ValueOf`:

```
func TypeOf(i any) Type          // returns the dynamic type of i
func ValueOf(i any) Value        // returns a Value holding the concrete value of i
```

```
package main
```

```go
import (
    "fmt"
    "reflect"
)

func main() {
    x := "Last Night"                    // Morgan Wallen song
    t := reflect.TypeOf(x)               // reflect.Type
    v := reflect.ValueOf(x)              // reflect.Value

    fmt.Println(t)                       // string
    fmt.Println(v)                       // Last Night
    fmt.Println(t.Kind())                // string
}
```

Both functions accept `any` (i.e., `interface{}`), so every value is automatically boxed before being inspected. That boxing is cheap for scalar values, but it does allocate for values that escape to the heap — one of the hidden costs of reflection.

`Kind` vs `Type`

`Type` and `Kind` are related but distinct.

`Type` is the specific named type as declared in your program — `string`, `int`, `Track`, `*Song`. `Kind` is the underlying category — `reflect.String`, `reflect.Int`, `reflect.Struct`, `reflect.Ptr`.

```go
type BPM int

b := BPM(140)
t := reflect.TypeOf(b)

fmt.Println(t)        // BPM
fmt.Println(t.Kind()) // int
```

`t.String()` returns `"BPM"` (the declared name); `t.Kind()` returns `reflect.Int` (the underlying kind). When you write a generic function using reflection — say, a printer that handles all integer-backed types — you switch on `Kind`, not on `Type`. If you switched on `Type`, you would need a separate case for every user-defined integer type.

Tip: In Java, `getClass()` is analogous to `Type`, and there is no direct equivalent of `Kind`. When writing reflection code that handles a family of types (all structs, all slices, all integer-backed types), switch on `Kind`.

The full set of kinds is:

```
reflect.Bool
reflect.Int, reflect.Int8, reflect.Int16, reflect.Int32, reflect.Int64
reflect.Uint, reflect.Uint8, reflect.Uint16, reflect.Uint32, reflect.Uint64, reflect.Uintptr
reflect.Float32, reflect.Float64
reflect.Complex64, reflect.Complex128
reflect.Array, reflect.Chan, reflect.Func, reflect.Interface
reflect.Map, reflect.Pointer, reflect.Slice, reflect.String
reflect.Struct, reflect.UnsafePointer
```

`reflect.Value.Elem()`

`Elem()` does two different things depending on the kind of value it is called on:

- On a **pointer**: it dereferences the pointer and returns the value the pointer points to.
- On an **interface**: it unwraps the interface and returns the concrete value inside.

```
func (v Value) Elem() Value  // dereference pointer or unwrap interface; panics if Kind is neither
```

This is the step that makes mutation possible. If you pass a plain value to `ValueOf`, you get a non-addressable copy — you cannot set its fields. If you pass a pointer, `Elem()` gives you the value the pointer points to, which is addressable and settable.

```
package main

import (
    "fmt"
    "reflect"
)

type Song struct {
    Title  string
    Artist string
    BPM    int
}

func main() {
    s := Song{Title: "Thought You Should Know", Artist: "Morgan Wallen", BPM: 144}

    // Inspect through a pointer so we can set fields later
    v := reflect.ValueOf(&s).Elem()          // Elem() dereferences the pointer

    fmt.Println(v.Type())                    // main.Song
    fmt.Println(v.Kind())                    // struct
    fmt.Println(v.FieldByName("Title"))      // Thought You Should Know
}
```

Wut: Calling `Elem()` on a value whose kind is not `Ptr` or `Interface` panics at runtime. Always check `v.Kind()` before calling `Elem()` in generic reflection code.

`reflect.Value.CanSet()`

Before writing to a `reflect.Value`, you must check `CanSet()`. A value is settable only if it was obtained by dereferencing a pointer — that is, the reflection object refers to the original memory location, not a copy.

```
func (v Value) CanSet() bool  // true if the value can be changed
func (v Value) Set(x Value)   // sets v to x; panics if !CanSet or types differ
```

```
x := 42
v := reflect.ValueOf(x)
fmt.Println(v.CanSet()) // false --- x was passed by value (a copy)

p := reflect.ValueOf(&x).Elem()
fmt.Println(p.CanSet()) // true  --- p refers to the original x
```

```
p.SetInt(99)
fmt.Println(x)          // 99
```

There are typed setters for each kind:

```
func (v Value) SetBool(x bool)
func (v Value) SetFloat(x float64)
func (v Value) SetInt(x int64)
func (v Value) SetString(x string)
func (v Value) SetUint(x uint64)
```

Trap: Unexported struct fields are never settable through reflection, even if you hold a pointer to the struct. `CanSet()` returns `false` for unexported fields. Attempting `Set()` on an unexported field panics. Always check `CanSet()` before writing.

Struct Field Iteration

One of the most common uses of reflection is walking the fields of a struct. `reflect.Type` provides methods for this:

```
func (t Type) NumField() int                        // number of fields
func (t Type) Field(i int) StructField              // field descriptor by index
func (t Type) FieldByName(name string) (StructField, bool) // field descriptor by name
```

A `reflect.StructField` carries the field's metadata:

```
type StructField struct {
    Name      string        // field name
    Type      Type          // field type
    Tag       StructTag     // raw struct tag string
    Index     []int         // index sequence for Type.FieldByIndex
    Offset    uintptr       // byte offset within the struct
    Anonymous bool          // true if this is an embedded field
    // unexported fields omitted
}
```

Here is a function that prints every exported field name, type, and value for any struct passed to it:

```
package main

import (
    "fmt"
    "reflect"
)

type Track struct {
    Title  string `display:"title"`
    Artist string `display:"artist"`
    BPM    int    `display:"bpm"`
}

func printFields(s any) {
    v := reflect.ValueOf(s)
    t := reflect.TypeOf(s)

    if t.Kind() != reflect.Struct {
```

```
        fmt.Println("not a struct")
        return
    }

    for i := range t.NumField() {
        field := t.Field(i)
        value := v.Field(i)
        fmt.Printf("%-10s %-8s = %v\n", field.Name, field.Type, value)
    }
}

func main() {
    tr := Track{Title: "First Class", Artist: "Jack Harlow", BPM: 97}
    printFields(tr)
}
```

Output:

```
Title      string   = First Class
Artist     string   = Jack Harlow
BPM        int      = 97
```

Tip: `for i := range t.NumField()` uses the Go 1.22 range-over-integer syntax, which is cleaner than `for i := 0; i < t.NumField(); i++`. Both produce the same result.

`reflect.StructField.Tag` and Custom Tag Parsing

Chapter 2 introduced struct tags as metadata for fields. Reflection is what makes custom tag parsing possible.

`reflect.StructField.Tag` is of type `reflect.StructTag`, a `string` alias with two methods:

```
func (tag StructTag) Get(key string) string                    // value for key, or "" if absent
func (tag StructTag) Lookup(key string) (value string, ok bool) // value + presence flag
```

Prefer `Lookup` over `Get` when the empty string is a meaningful tag value — `Get` cannot distinguish "absent" from `""`.

```
type Track struct {
    Title    string `json:"title"         display:"Title"`
    Artist   string `json:"artist"        display:"Artist"`
    BPM      int    `json:"bpm,omitempty" display:"BPM"`
    internal string // unexported --- skip in generic code
}

func printTagged(s any) {
    t := reflect.TypeOf(s)
    if t.Kind() != reflect.Struct {
        return
    }
    for i := range t.NumField() {
        field := t.Field(i)
        if !field.IsExported() {
            continue
        }
        label, ok := field.Tag.Lookup("display")
```

```
        if !ok {
            label = field.Name // fall back to field name
        }
        fmt.Printf("%s: %v\n", label, reflect.ValueOf(s).Field(i))
    }
}
```

`field.IsExported()` (added in Go 1.17) is the clean way to skip unexported fields. Before 1.17 the idiom was `field.PkgPath == ""`, which still works but is less readable.

Tip: Tag values often carry options after a comma (e.g., `json:"bpm,omitempty"`). The `reflect.StructTag` methods return the full value string including the comma and options. Use `strings.Cut(value, ",")` to separate the name from the options:

```
raw, _ := field.Tag.Lookup("json")
name, _, _ := strings.Cut(raw, ",")
```

When to Use Reflection

Reflection is warranted when the types involved are genuinely unknown at compile time. The two canonical cases in production Go are:

Serialization libraries. `encoding/json`, `encoding/xml`, and `encoding/gob` all use reflection to marshal and unmarshal arbitrary user-defined types. They read struct tags, walk fields, and convert between Go values and encoded bytes — none of which is possible without reflection, because the library authors cannot know your struct in advance.

Dependency injection frameworks. Frameworks like `google/wire` (compile-time code generation) and `uber-go/fx` (runtime DI) use reflection to inspect constructor function signatures, discover what types they produce and consume, and wire them together automatically.

Other valid uses include:

- Test assertion libraries (`github.com/stretchr/testify`) that deep-compare arbitrary values.
- ORM packages that map struct fields to database columns using `db:"..."` tags.
- Configuration loaders that populate structs from environment variables or config files.
- Code generators that read struct definitions to emit boilerplate (e.g., `stringer`, `protoc-gen-go`).

Trap: Reflection is often reached for when generics or interfaces would work just as well. If you know the set of types at compile time, a generic function or a narrow interface is faster, safer, and more readable. Reach for reflection only when the type truly cannot be known until runtime. Chapter 17 covers when generics are the right choice instead. [*no-premature-interface*]

Performance Cost of Reflection

Reflection is significantly slower than direct code. Every `TypeOf`, `ValueOf`, field access, and setter call goes through the runtime's type system rather than being resolved at compile time. The costs add up quickly:

- **Interface boxing** — passing a value as `any` to `ValueOf` may allocate.
- **No inlining** — reflect calls cannot be inlined by the compiler.
- **Dynamic dispatch** — field accesses are index lookups, not direct memory offsets.
- **No static type checking** — type errors become runtime panics, not compile errors.

A rough rule of thumb: reflection-based code is typically 10–100x slower than equivalent direct code. For hot paths in a service, that difference is material.

```go
// direct field read --- nanoseconds
title := track.Title

// reflected field read --- tens to hundreds of nanoseconds
title := reflect.ValueOf(track).FieldByName("Title").String()
```

Mitigation strategies:

- Cache `reflect.Type` values at startup — calling `reflect.TypeOf` on the same type repeatedly re-derives the type descriptor each time; storing it in a package-level variable avoids that cost.
- Cache field index paths — `Type.FieldByName` does a linear scan; computing the index once and using `Type.Field(i)` on subsequent calls is faster.
- Consider code generation — tools like `easyjson` and `msgp` generate direct marshal/unmarshal code from struct definitions, eliminating runtime reflection entirely.

Tip: The standard library's own `encoding/json` is noticeably slower than generated alternatives for the same reason. If JSON throughput matters in your service, benchmark both approaches before committing to a design. See Chapter 18 for how to write benchmarks.

Prefer Generics or Interfaces

Before reaching for `reflect`, ask yourself:

1. **Can an interface solve it?** If the operation can be expressed as a method set, define an interface. Types opt in by implementing the methods, and the compiler enforces correctness. Return the concrete type from implementing packages rather than the interface itself, so callers can always add methods later. [*return-concrete-types*]
2. **Can a generic function solve it?** If you need to handle a fixed family of types uniformly (e.g., "all ordered types", "all slices"), a generic function (Chapter 17) works at compile time with no runtime overhead.
3. **Can code generation solve it?** If the types are known at compile time but the boilerplate is too tedious to write by hand, `//go:generate` with a code generator is better than runtime reflection. The generated code is explicit, fast, and debuggable.

Reflection becomes the right answer when none of the above apply — when the struct definition is only available at runtime (read from a plugin, received as user input, or declared in a package your code never imports directly).

Wut: In Java, reflection is a common idiom even for things that could be expressed with generics or interfaces, partly because Java generics were bolted on later and have significant limitations (type erasure, no primitive type parameters). Go generics (1.18) are more capable than Java generics in several ways (no erasure, can range over type parameters), so the case for reflection is narrower in Go than it is in Java.

Key Points

- Reflection has three laws: interface -> reflect object, reflect object -> interface, and mutation requires settability (i.e., a pointer).
- `reflect.TypeOf(x)` returns the `reflect.Type` describing the dynamic type of `x`; `reflect.ValueOf(x)` returns a `reflect.Value` holding its value.
- `Kind()` returns the underlying category (`reflect.Struct`, `reflect.Ptr`, etc.); `Type()` returns the specific named type. Switch on `Kind` when writing generic reflection code.

- `Elem()` dereferences a pointer or unwraps an interface; it panics if the kind is neither.
- `CanSet()` must be true before calling any setter; values are settable only when obtained through a pointer.
- Unexported fields are never settable through reflection.
- Walk struct fields with `Type.NumField()` and `Type.Field(i)`; skip unexported fields with `StructField.IsExported()`.
- `StructField.Tag.Lookup(key)` retrieves a tag value; prefer it over `Get` when the empty string is meaningful.
- Reflection is the right tool for serialization libraries, dependency injection, and test assertion frameworks — cases where the concrete type is genuinely unknown at compile time.
- Reflection is 10–100x slower than direct code; cache `reflect.Type` values and field indices, or use code generation for hot paths.
- Before reaching for reflection, consider: can an interface solve it? can a generic function solve it? can code generation solve it?

Exercises

1. **Think about it:** Both Java's `java.lang.reflect` and Go's `reflect` package let you inspect types and values at runtime. Name two ways they are fundamentally similar and two ways they differ. In what situation would you reach for reflection in a Go program where a Java programmer might have reached for it as well, and in what situation would a Go programmer choose generics or interfaces instead?

2. **What does this print?**

```go
package main

import (
    "fmt"
    "reflect"
)

type Album struct {
    Title  string
    Artist string
    Tracks int
}

func main() {
    a := Album{Title: "Jackman", Artist: "Jack Harlow", Tracks: 13}
    t := reflect.TypeOf(a)
    v := reflect.ValueOf(a)

    for i := range t.NumField() {
        f := t.Field(i)
        fmt.Printf("%s (%s): %v\n", f.Name, f.Type.Kind(), v.Field(i))
    }
}
```

3. **Calculation:** Trace through the following program step by step. What does it print?

```go
package main

import (
    "fmt"
    "reflect"
```

```
)

func main() {
    n := 97
    v := reflect.ValueOf(n)
    fmt.Println(v.CanSet())

    p := reflect.ValueOf(&n).Elem()
    fmt.Println(p.CanSet())

    p.SetInt(99)
    fmt.Println(n)

    s := "Last Night"
    sv := reflect.ValueOf(&s).Elem()
    sv.SetString("First Class")
    fmt.Println(s)
}
```

4. **Where is the bug?** The following function is supposed to double every `int` field in a struct. It compiles and runs without panicking, but the original struct is never modified. Why?

```
package main

import (
    "fmt"
    "reflect"
)

type Stats struct {
    Plays int
    Likes int
}

func doubleInts(s any) {
    v := reflect.ValueOf(s)
    t := v.Type()
    for i := range t.NumField() {
        f := v.Field(i)
        if f.Kind() == reflect.Int && f.CanSet() {
            f.SetInt(f.Int() * 2)
        }
    }
}

func main() {
    stats := Stats{Plays: 500_000, Likes: 12_000}
    doubleInts(stats) // bug is here
    fmt.Println(stats)
}
```

5. **Write a program:** Implement a function `StructToMap(s any) map[string]any` that uses reflection to convert any struct's exported fields into a `map[string]any`. The map keys should come from a `map:"key"` struct tag if present, and fall back to the field name otherwise. Ignore unexported fields. Test it on a struct with at least three exported fields and one unexported field, with at least two fields

having `map:"..."` tags. Print the resulting map.